Skuul — Feature Guide, Run Guide & Upgrade Master Prompt

Nov 4, 2026 · @Eker

> **Update 5 Nov 2026 — the app is now Campuly.** The brand is applied in the code, and the current build plan is **Master Prompt v2** (`MASTER_PROMPT.md` in the Campuly source package). It adds exact per-step instructions, a database-portability step for PostgreSQL/Supabase and SQLite, and the full `docs/` set. The prompt at the bottom of this page is v1, kept for reference.

Summary

Skuul (V3, `main` branch) is a serious, production-grade, multi-campus school management system built on Laravel 13 + Livewire 4. I installed it from your zip, ran all 215 migrations, loaded the official demo school, and logged in as platform admin, teacher and parent: every screen I opened returned HTTP 200 and rendered correctly.

The core is strong; the three things you asked for are where it is weakest:

- **French is listed but not implemented.** `config/app.php` offers 15 languages including `fr`, but `lang/` only holds English, and 167 of 168 Livewire views contain no `__()` translation call. Switching the installation to French changes only `<html lang>`; every word stays English. Language is also install-wide, not per user.
- **No PWA.** There is no web manifest and no service worker.
- **No API.** `routes/` has only `web.php` and `console.php`. A desktop/offline client has nothing to sync against yet, so an API layer is the first building block of the hybrid app.

The master prompt at the end of this doc is phased so an AI coding agent (Claude Code, Cursor, etc.) can deliver these upgrades step by step without breaking the existing modules.

Architecture and how it works

Skuul is a server-rendered monolith: every screen is a Blade view or a Livewire component, and all business rules live in PHP Action and Service classes behind Laravel policies.

Layer	What it uses	Size in this codebase
Language / framework	PHP 8.4+, Laravel 13	composer.lock pinned
UI	Livewire 4, Blade, Alpine (via Livewire), Tailwind CSS 4, the April UI kit (`yungifez/april-ui`), Lucide icons	434 Blade views, 168 Livewire components
Business logic	`app/Actions/*` (one class per operation) and `app/Services/*`	~150 actions/services in 30+ domains
Data	MySQL 8, Eloquent	129 models, 215 migrations, 68 enums
Access control	Spatie Permission + 55 policy classes + custom campus roles	roles: platform-admin, organization-admin, admin, teacher, student, parent, plus custom (accountant, librarian…)
Auth	Laravel Fortify + Jetstream (2FA, passkeys), invite-only accounts	no public sign-up
Background work	Redis queue, scheduler (16 scheduled tasks)	notices, backups, library holds, calendar advance
Documents	DomPDF or a headless-browser renderer	report cards, transcripts, receipts, timetables
Tests	Pest 5	193 test files

Tenancy model. Organization → School (campus) → Academic year → Academic period (term/semester). Every staff request runs through middleware that resolves the domain, sets the *working school* and the *working period*, and checks the account is active. Most routes are wrapped in `feature:` middleware, so a school can switch whole modules off.

Request flow. Browser → `routes/web.php` → middleware (auth, verified, active account, password change, working school, period, feature flag) → controller returns a page → Livewire component on that page handles forms and tables over AJAX → Action class validates and writes inside a DB transaction → audit event recorded → policy-filtered result re-rendered.

Families (students and parents) use a separate *portal* route group scoped by enrolment, so a parent only ever sees their own children's records.

Setup flow. A browser installer creates the admin, organization and first campus; a school setup checklist (10 steps) then walks through school year, terms, teaching approach, classes, sections and grading scales; an academic-year wizard publishes each new year.

All features, module by module

Skuul ships 23 modules; the ones marked *toggle* can be switched off per campus under School settings → Features.

Module	What it does
Organizations and campuses	Multi-organization, multi-campus; org dashboard, members, custom domains per school, billing groups, shared calendar templates, shared boarding residences, campus-move requests, record-sharing requests between schools
School setup and “school language”	10-step setup checklist; operating profile; presets that rename terms on screen (e.g. “Grade” vs “Form”, “Term” vs “Semester”) — note this renames labels, it is not translation
Academic structure	Academic years (with setup wizard, publish, roll forward), periods (terms/semesters with open/close status), classes/levels, sections, cohorts, instructional model (homeroom vs subject teaching), automatic calendar advance
People	Students, teachers, admins, parents; admission numbers; profiles with print view; invitations by email (72 h expiry); bulk import of students and staff from files (*toggle: imports*)
Admissions	Waitlist board with offer / accept / decline
Enrolment lifecycle	Promotions, graduations, leavers losing access automatically, enrolment status history, transfer packages

Module	What it does
Subjects and course offerings	Subjects across levels, course offerings with rosters, bulk create, roll forward, teaching assignments
Gradebook and results	Assessment templates, grade categories and items, mark sheets, grading scales (letters, points, %), result approval, result snapshots, report cards, transcripts, rankings (*toggle*)
Exams	Exam planning per calendar, active/inactive exams
Attendance (*toggle*)	Daily and per-lesson registers, change history, dashboard trend, family view
Timetables	Weekly timetables, time slots, custom items, substitutions/cover panel, print, a personal week view
Syllabus and lesson notes	Syllabi with topics, coverage tracker and report, lesson-note book, shared syllabus library, “behind schedule” reminders
Finance	Fee categories, fees, invoices and invoice batches, payments with allocation, receipts, student accounts, expenses, cash deposits, budgets, financial periods, full double-entry ledger; channels: cash, bank transfer, cheque, card, mobile money (recorded manually), Stripe checkout (online)
Financial reports	Balance sheet, income statement, trial balance, general ledger, cash & bank summary, budget variance, income by fee type, student balances, student aging, expenses; plus class lists
Student care	Incidents and safeguarding (*toggle: discipline*); support plans, health records with separate access (*toggle: wellbeing*)
Staff operations (*toggle*)	Staff profiles, credentials, availability, leave requests with approval history

Module	What it does
Calendar and notices	Calendar events with audiences (*toggle: events*); notices with attachments, scheduled sending and per-user email preferences
Boarding (*toggle*)	Houses, rooms, beds, supervisors, roll calls, nights-away (overnight leave) desk
Library (*toggle*)	Catalogue titles and copies, lending desk, reservations queue with automatic holds, lending rules
Facilities	Rooms/facilities and bookings
Programmes and graduation plans (*toggles*)	Clubs/activities with participation; graduation requirements, exemptions and progress
Family portal (*toggle*)	Per child: results, attendance, timetable, calendar, notices, invoices and payments, documents (report-card and transcript downloads), requests to the school, library, boarding, graduation progress, programmes, syllabi
Platform and operations	Health endpoint, encrypted backups with retention, restore rehearsals, backup checks, queue heartbeat, log viewer, in-app updater, demo mode, release-readiness command, audit log
Experience	Light/dark/system theme, collapsible sidebar, Ctrl+K command palette, fullscreen toggle, skip-to-content link, responsive layout

How to run it

This exact sequence got it running end-to-end in my test environment (PHP 8.4.27, MySQL 8.0, Node 22, Redis). You need **PHP 8.4.1+, Composer 2, Node.js 22–24, MySQL 8** and, optionally, Redis.

Option A — Windows without Docker (simplest for you)

1. Install Laravel Herd for Windows (gives PHP 8.4 + Composer) and Node.js LTS. Install MySQL 8 (Herd Pro includes it; otherwise MySQL Installer or Laragon). Create an empty database `skuul`.
2. Unzip `skuul-main` into a folder, open a terminal there.
3. Run:

```
composer install --no-interaction --prefer-dist
npm ci
npm run build
copy .env.example .env
php artisan key:generate
```

4. Edit `.env`: `DB_HOST=127.0.0.1`, `DB_USERNAME` / `DB_PASSWORD` = your MySQL user, `APP_URL=http://127.0.0.1:8000`, `APP_CURRENCY=XAF`, `MAIL_MAILER=log`. If you skip Redis, set `QUEUE_CONNECTION=database` (or `sync` for testing).
5. Create the tables and reference data:

```
php artisan migrate
php artisan db:seed --class=WorldSeeder
php artisan storage:link
php artisan serve
```

6. Open http://127.0.0.1:8000/install, create your admin, organization and first campus, then complete the setup checklist.
7. For notices, reminders and backups to work, keep two more terminals running: `php artisan queue:work --tries=3` and `php artisan schedule:work`.

Option B — Docker Desktop + Sail (the project's official way)

Needs Docker Desktop with WSL2. Run `composer install`, `cp .env.example .env`, then `vendor/bin/sail up -d` followed by the README's Sail commands (`sail npm ci`, `sail npm run build`, `sail artisan key:generate`, `migrate`, `db:seed --class=WorldSeeder`, `storage:link`).

Exploring with the demo school (disposable database only)

Set `DEMO_MODE=true` in `.env`, run `php artisan config:clear` then `php artisan skuul:refresh-demo`. Every demo account's password is `password`:

Role	Email
Platform admin (sees everything)	megan.carter@staff.riversideusd.example
School admin	karen.whitfield@staff.riversideusd.example
Accountant	james.patel@staff.riversideusd.example
Librarian	emily.rodriguez@staff.riversideusd.example
Teacher	angela.torres@staff.riversideusd.example
Parent	abigail.vasquez@family.riversideusd.example
Student	abigail.sanders@student.riversideusd.example

Demo mode wipes data every hour and refuses deletions. Never run it, or the default `db:seed`, on a production database.

Deploying online

Any VPS or Laravel host with PHP 8.4, MySQL 8 and Redis (Forge, Ploi, a Hostinger VPS, or Sevalla). Run `composer install --no-dev --optimize-autoloader`, `npm run build`, `php artisan migrate --force`, `php artisan optimize`, a Supervisor process for `queue:work`, and a cron entry `* * * * * php artisan schedule:run`. Set `APP_ENV=production`, `APP_DEBUG=false`, HTTPS, and a `BACKUP_KEY` (backups refuse to run without one).

Gaps found during the audit

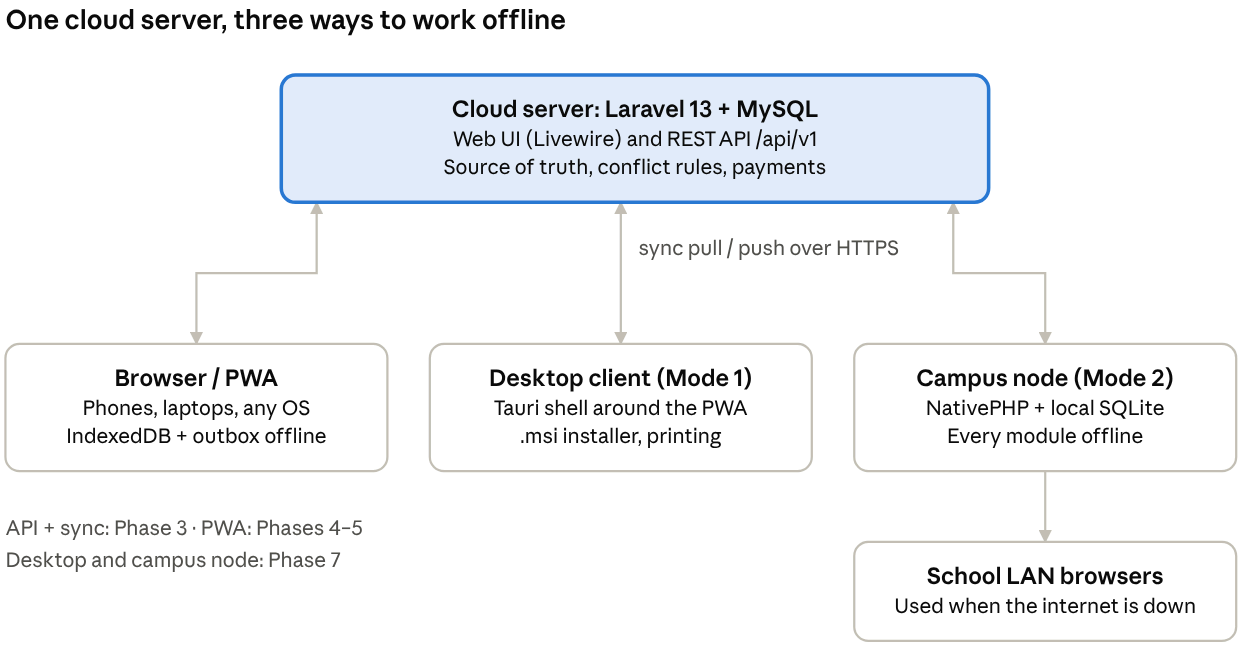
These are what stand between the current code and the app you described, ordered by how much they block it.

Gap	Evidence in the code	Impact
Translation not wired	`lang/` has only `en/` + a 7-line `en.json`; 167 of 168 Livewire views have no `__()`; 68 enums return English labels from PHP; 166 hard-coded `->format()` dates	French cannot be offered without touching nearly every view

Gap	Evidence in the code	Impact
Language is install-wide	`SetApplicationLocale` reads one locale from the `installations` table	No per-user switch, no instant toggle
No API	Only `routes/web.php` and `routes/console.php`; Sanctum is installed but unused	Desktop/mobile/offline clients have nothing to sync with
No PWA	No manifest, no service worker	Not installable, nothing works offline
Livewire is online-only	Every form and table round-trips to the server	Offline mode needs a separate client-side data layer, not just caching
Cameroon payments	Mobile money is a manual “recorded by the office” channel; Stripe is the only online gateway	No MTN MoMo / Orange Money collection
Messaging	Email notices only; no SMS, WhatsApp or in-app chat	Weak reach for parents without email
Teaching tools	No homework/assignments, online quizzes or e-learning content	Gradebook exists, but no LMS layer
Visual design	Clean but plain, low-contrast sage palette; tall table rows; mobile is usable but desktop-first	The UI overhaul has a solid accessible base to build on
Release status	V3 is pre-release; migrations from V2 are destructive	Fork from a fixed commit and pin it; never upgrade a live V2 database in place

Master upgrade prompt, step by step (v1, superseded by v2)

The same prompt is attached as a Markdown file for pasting into Claude Code, Cursor or Codex. It runs in 10 phases (0–9); the order matters because the API (Phase 3) is what the PWA offline layer, the sync engine and the desktop app all build on.



target architecture · cloud + 3 offline modes

Every client talks to the same cloud API; each keeps its own offline store and syncs through one protocol, so the conflict rules are written once.

> Paste the **Context** and **Global rules** blocks at the start of every session with your AI coding agent. Then give it **one phase at a time**. Do not move to the next phase until every acceptance check of the current one passes.

CONTEXT (paste every session)

You are working on **Skuul V3**, an open-source school management system (MIT) that I am forking and rebranding for schools in Cameroon and francophone/anglophone Africa.

Stack (do not change without asking me): PHP 8.4, Laravel 13, Livewire 4, Blade, Alpine, Tailwind CSS 4, April UI kit (`yungifez/april-ui`), Lucide icons, MySQL 8, Redis, Fortify/Jetstream (2FA, passkeys), Spatie Permission, DomPDF, Pest 5.

Codebase facts:

- 129 Eloquent models, 215 migrations, 68 enums, 168 Livewire components, 434 Blade views, 86 controllers, 55 policies, ~150 Action/Service classes in `app/Actions` and `app/Services` , 193 Pest test files.

- Tenancy: Organization → School (campus) → Academic year → Academic period. Middleware sets the *working school* and *working period*. Modules are switchable per campus via `App\Enums\Feature` and `feature:` route middleware.
- Roles: platform-admin, organization-admin, admin, teacher, student, parent, plus custom campus roles (accountant, librarian…).
- Family portal routes live under `dashboard/portal/...`, scoped by enrolment.
- Language today: `config/app.php` lists 15 locales but `lang/` only has English; 167 of 168 Livewire views have no `__()`; enum `label()` methods return hard-coded English; ~166 hard-coded `->format()` date calls in views. Locale is install-wide (`SetApplicationLocale` reads `installations.locale`).
- No API (`routes/api.php` does not exist; Sanctum installed but unused). No PWA manifest or service worker.
- Payments: cash, bank, cheque, card, mobile money (manual record only), Stripe checkout.
- Read `AGENTS.md`, `CLAUDE.md` and `.ai/rules` in the repo and follow them.

GLOBAL RULES (paste every session)

1. Work in small, reviewable commits on a feature branch per phase. Never commit to `main` directly.
2. Before changing anything, read the files you will touch and their siblings; follow existing conventions (Action classes, policies, Livewire patterns, enums).
3. Keep every existing feature working. Run `php artisan test --compact`, `vendor/bin/pint --dirty`, `vendor/bin/phpstan analyse` and `npm run build` at the end of each step; fix what you broke before continuing.
4. Write or update Pest tests for every behaviour you add. New UI flows get a feature test; new API endpoints get request tests including authorization failures.
5. **Every user-visible string you add or touch must go through the translator** (`__()`, `trans_choice()`, `@lang`, or the JS `t()` helper) with keys in BOTH `lang/en.json` and `lang/fr.json`. No exceptions: buttons, placeholders, tooltips, aria-labels, toasts, validation messages, emails, PDFs, enum labels, empty states, error pages, JS alerts, the manifest, the installer.
6. Never weaken authorization: every new endpoint and component goes through the existing policies.
7. No destructive migrations. Additive columns only, with backfills, and every migration must run on MySQL 8 **and** SQLite (needed for the desktop app in Phase 7).

8. When a decision is genuinely mine (branding, payment provider contracts, hosting), stop and ask me with 2–3 options and your recommendation.
9. At the end of each phase, give me: what changed, how to test it manually in 5 steps, and the list of anything left undone.

PHASE 0 — Baseline and fork hygiene

Goal: a clean, reproducible starting point.

1. Pin the fork to the current commit; tag it `upstream-base`.
2. Get the full test suite green on PHP 8.4 + MySQL 8 locally. Record the baseline (tests passed, PHPStan level, build size) in `docs/upgrade/BASELINE.md`.
3. Add a SQLite testing profile (`.env.testing.sqlite`) and run the suite against it. List every migration or query that fails on SQLite in `docs/upgrade/SQLITE_GAPS.md` (do not fix yet).
4. Set up GitHub Actions to run tests, Pint, PHPStan and `npm run build` on every push.
5. Rebrand config only (no UI work yet): `APP_NAME`, logo path, favicon set, default currency `XAF`, default timezone `Africa/Douala`, default country Cameroon in the installer.

Acceptance: CI green; baseline doc and SQLite gap list committed; app installs and runs with the new name and XAF.

PHASE 1 — Full bilingual foundation (English ⇄ French)

Goal: every word in the product can appear in English or French, and any user switches instantly.

1.1 Locale model

- Keep only `en` and `fr` in `config/app.supported_locales` (others can return later). Add `APP_DEFAULT_LOCALE` (default `fr`).
- Add `users.locale` (nullable, `en|fr`) and `schools.default_locale`.
- Rewrite `SetApplicationLocale` with this priority: `?lang=` query → session → authenticated user's `locale` → school default → installation default → browser `Accept-Language` → `fr`.
- Persist the choice to the user (if signed in), the session and a long-lived cookie; set Carbon locale, `NumberFormatter` locale (`fr_CM` / `en_CM`) and the `<html lang>` attribute together.

1.2 Switcher

- Language switcher (EN | FR segmented control labelled "English" / "Français") in: the top header, the user menu, the login/forgot-password/invitation pages, the installer, and the family portal. Keyboard accessible, `aria-pressed`.
- Switching sends a POST to `/locale`, updates the stores above and reloads the current URL in under 300 ms (no logout; persist unsaved Livewire form fields in session before reload or warn the user).
- Add a keyboard shortcut `Alt+L` and a command-palette entry "Switch language / Changer de langue".

1.3 Externalise every string

Work module by module, one commit per module, in this order: layouts & navigation → auth & installer → dashboard → people → academic structure → gradebook/results → attendance → timetable → finance → reports/PDFs → portal → care modules → boarding/library/facilities → settings → emails/notifications → errors.

- Wrap every literal in Blade and Livewire PHP (`session()->flash`, validation `attributes`, `confirm` dialogs, table headers, column labels, toasts).
- Enums: change every `label()` / `description()` / `group()` to return `__('enum.feature.attendance')`-style keys. Keep the enum *values* unchanged.
- Dates: replace hard-coded `->format('D, M j, Y')` with a helper `local_date($date, 'medium')` that uses `isoFormat` with locale-aware presets (`short`, `medium`, `long`, `time`, `datetime`). French output example: « mer. 4 nov. 2026 ».
- Numbers & money: one helper `money()` and `number()` using `NumberFormatter` with the active locale; XAF shows as `125 000 FCFA` in French and `FCFA 125,000` in English; XAF has no decimals.
- Pluralisation with `trans_choice`. Gendered French agreements handled in the French strings, not in code.
- Validation: add complete `lang/fr/validation.php`, `auth.php`, `passwords.php`, `pagination.php`, plus French attribute names for every form field.
- The existing "school language" presets (labels such as Grade/Form, Term/Semester) become **bilingual**: each label stored as `{ "en": "...", "fr": "..." }` with presets for the Cameroonian systems (anglophone: Form 1–5, Lower/Upper Sixth; francophone: 6e–Terminale, trimestres/séquences).

- JavaScript: create `resources/js/i18n.js` exposing `t(key, params)`; inject the active locale's JS keys into the layout as JSON. Translate the command palette, theme toggle, date pickers and any Alpine text.
- PDFs (report cards, transcripts, receipts, invoices, timetables) render in the **recipient's** language, with an option to print bilingual (EN/FR side by side) report cards.
- Emails and notifications are sent in each recipient's own locale (`HasLocalePreference` on `User`).
- Error pages (403/404/419/500/503), the maintenance page and the installer are bilingual.

1.4 Quality gates

- A Pest test that scans all Blade/Livewire files for visible text nodes not wrapped in a translation call and fails with a list (allow-list for brand names).
- A test that `lang/en.json` and `lang/fr.json` have exactly the same keys and no empty values.
- A Playwright smoke test that visits 40 key screens in both languages and fails if any known English-only word appears on a French page.
- French copy must be natural, professional French (« Élèves », « Enseignants », « Parents d'élèves », « Bulletins », « Frais de scolarité », « Emploi du temps », « Appel », « Trimestre »). Produce a glossary `docs/i18n/GLOSSARY.md` (EN → FR, 300+ domain terms) first and use it consistently.

Acceptance: with French selected, I can click through every menu item as admin, teacher, accountant, parent and student and see no English; switching back and forth takes under a second; report card PDF prints in both languages; all quality-gate tests pass.

PHASE 2 — New design system and complete UI/UX overhaul

Goal: a modern, distinctive, fast, accessible interface that works equally well on phones, tablets and desktops.

2.1 Design system first

- Before coding screens, propose to me 3 visual directions (name, palette, typography, one dashboard mockup each). Wait for my choice.

- Implement the chosen direction as design tokens in Tailwind 4 `@theme` (colours with light + dark values, radius, spacing, shadow, motion durations, z-index), re-theming April UI components rather than replacing them.
- Typography: one variable sans for UI (e.g. Inter / Plus Jakarta Sans / Manrope) with a clear 6-step type scale; tabular numerals for all figures.
- Contrast: WCAG 2.2 AA minimum everywhere. Colour is never the only carrier of meaning.
- Document every component in a `/dev/ui` page (local env only): buttons, inputs, selects, date pickers, tables, cards, stat tiles, badges, tabs, modals, drawers, toasts, empty states, skeletons, charts.

2.2 Layout and navigation

- Collapsible sidebar grouped by role, with icons and counters (e.g. pending requests); remembered state.
- Mobile: bottom navigation bar with the 4–5 most used items per role + "More"; tables become stacked cards under 768 px; sticky action bars on forms.
- Global search (Ctrl/Cmd+K) across students, staff, parents, invoices, classes, with recent items and quick actions ("Record payment", "Take attendance").
- Header: school switcher, period switcher, language switcher, theme, notifications bell, online/offline/sync indicator (Phase 5), profile.
- Breadcrumbs, page titles and primary actions placed consistently.

2.3 Role-based dashboards (redesign all)

- **Admin/Director:** enrolment, attendance today vs 4-week trend, fees collected vs expected (by class), overdue invoices, upcoming events, setup health, recent activity.
- **Teacher:** today's timetable, registers to take, mark sheets due, syllabus coverage behind, my classes.
- **Accountant/Bursar:** collections today/week/month, by channel (cash, MoMo, Orange Money, bank), top debtors, aging buckets, deposits to bank.
- **Parent/Student:** per child — attendance %, latest results, balance due with "Pay now", timetable today, notices.
- Use a single chart library (Apache ECharts or Chart.js) wrapped in a Blade component that reads the active locale and theme.

2.4 UX patterns everywhere

- Skeleton loaders and optimistic UI for Livewire actions; never a blank page.
- Inline validation with French/English messages; autosave drafts for long forms (mark sheets, student registration).
- Clear empty states with a primary action; confirmation dialogs that name the object ("Delete invoice FAC-2026-0042?").
- Bulk actions on tables (select rows → export, message, print, promote).
- Exports: CSV/Excel/PDF on every list.
- Toasts with undo for reversible actions.
- Print styles for every printable page.
- Accessibility: focus rings, keyboard navigation for tables and the timetable grid, `aria-live` for toasts, reduced-motion support.

Acceptance: Lighthouse ⩾ 90 for Accessibility and Best Practices on the dashboard, students list and portal (mobile and desktop); all screens reviewed at 360, 768, 1280 and 1920 px; dark mode complete; no regressions in tests.

PHASE 3 — Versioned REST API (foundation for PWA offline, desktop and mobile)

Goal: a secure, documented API that exposes the same rules as the web app.

1. Create `routes/api.php` with prefix `/api/v1`, Sanctum token auth (device-named tokens, revocable from the profile page), rate limiting per token.
2. Reuse Actions and Policies — controllers stay thin. API Resources for every exposed model; consistent envelope `{ data, meta, links }`; errors in RFC 9457 problem+json with translated `title` / `detail` (honour `Accept-Language`).
3. Endpoints (minimum): auth (login, logout, me, tokens), schools & periods, classes/sections, students, staff, parents, timetable, attendance (read + write registers), gradebook (read + write marks), fees/invoices/payments (read + record payment), notices, calendar, portal endpoints for families, library loans, reports list/download.
4. Every syncable table gets: `uuid` (ULID, unique, indexed, backfilled), `updated_at` indexed, soft deletes (`deleted_at`) as tombstones, and a `version` integer. Clients may create records with their own ULID.
5. **Sync endpoints:**

- `GET /api/v1/sync/pull?since=<cursor>&scopes=...` → changed + deleted records the user may see, paginated, with a new cursor.
- `POST /api/v1/sync/push` → batch of mutations `{ client_mutation_id, entity, uuid, op, base_version, payload }`, idempotent (store processed mutation IDs), each validated through the same Action classes; returns per-item `applied | conflict | rejected` with the server copy on conflict.

6. OpenAPI 3.1 spec generated (e.g. Scramble) at `/docs/api`, bilingual descriptions.
7. Webhooks out (optional): payment received, invoice overdue, attendance absent.

Acceptance: API test suite covers auth, policy denials per role, pagination, sync pull/push round-trip, idempotency (same mutation sent twice applies once) and conflict responses.

PHASE 4 — Progressive Web App (installable on desktop and phone)

Goal: the app installs from the browser on Windows, macOS, Linux, Android and iOS, starts fast and opens even without network.

1. Web app manifest generated per locale (`name`, `short_name`, `description` translated), theme/background colours from tokens, maskable icons (192, 512), shortcuts (Take attendance, Record payment, My children), screenshots for the install dialog, `display: standalone`, `start_url: /dashboard?source=pwa`.
2. Service worker via `vite-plugin-pwa` (Workbox):
   - Precache the app shell (CSS, JS, fonts, icons, offline page).
   - Runtime caching: static assets cache-first; API GETs and key pages stale-while-revalidate; never serve one user's authenticated HTML to another (key caches by user id; clear on logout).
   - Bilingual offline fallback page.
   - Update flow: "A new version is available — Reload" toast (translated).
3. Custom install button in the header/user menu using `beforeinstallprompt`, with an iOS "Add to Home Screen" instruction sheet.
4. Web push notifications (VAPID) for notices, absences and payment confirmations, sent in the user's language, opt-in per user.

Acceptance: Lighthouse PWA checks pass; the app installs on Chrome/Edge desktop with its own window and icon; airplane-mode reload shows the offline page instead of a browser error.

PHASE 5 — Offline-first data layer and sync engine

Goal: core daily work continues with no internet and syncs automatically when the connection returns.

1. Client store: IndexedDB via Dexie in `resources/js/offline/`. Tables mirror the sync scopes the signed-in user may see (their school, period, classes, children).
2. Outbox: every offline write is stored as a mutation (Phase 3 format) and shown in a "Pending sync (n)" panel the user can review.
3. Sync worker: runs on startup, on the `online` event, every 2 minutes while online, and via Background Sync where supported; exponential back-off; progress and last-sync time shown in the header indicator (green online · amber syncing · grey offline · red conflicts).
4. **Offline-capable screens** (Alpine/JS views reading IndexedDB, falling back to Livewire when online):
   - Attendance register (daily and per lesson)
   - Mark entry / gradebook mark sheets
   - Student, parent and staff directory + profiles (read)
   - Timetable and personal week (read)
   - Notices and calendar (read)
   - Record a payment and print a provisional receipt (number `DEVICE-YYYY-#####`, replaced by the official number after sync)
   - Library check-out/check-in
   - Boarding roll call
   - Family portal: results, attendance, timetable, balance (read)
   - Everything else shows a clear, translated "Available online only" state, not an error.
5. Conflict policy (document in `docs/sync/CONFLICTS.md` and implement):
   - Attendance: last-writer-wins per (student, date, lesson), full history kept in `attendance_changes`.
   - Marks: field-level; if the server value changed since `base_version`, flag a conflict for the teacher to resolve in a side-by-side dialog.
   - Payments: append-only, never overwritten; duplicates detected by (student, amount, channel, reference, date).
   - Structure (years, classes, fees, users, permissions): server-authoritative, read-only offline.

6. Security: encrypt the IndexedDB payload with a key derived after login (WebCrypto); wipe local data on logout or after 30 days without sync; respect role scopes.

Acceptance: with network disabled, a teacher takes attendance for 3 classes and enters marks; an accountant records 5 payments and prints receipts; on reconnect everything syncs within 2 minutes and appears on another device, and a deliberately conflicting mark shows the resolution dialog. An automated Playwright test covers this scenario.

PHASE 6 — Cameroon-ready payments and messaging

1. Mobile money collection through an `OnlinePaymentGateway` implementation (the interface already exists for Stripe):
   - Option A (recommended to start): an aggregator covering MTN MoMo **and** Orange Money in one integration (e.g. CamPay, NotchPay, Flutterwave — ask me which I have a merchant account with).
   - Option B: direct MTN MoMo Open API (Collection, target environment `mtncameroon`) + Orange Money Web Payment API.
   - Request-to-pay from the parent portal ("Pay now"), webhook + polling confirmation, automatic allocation to invoices, bilingual SMS/email receipt.
2. SMS and WhatsApp channels for notices, absence alerts, fee reminders and results release (provider adapter interface; ask me for the provider). Per-user channel preferences, all templates in EN and FR.
3. In-app messaging between parents and teachers/administration with read receipts and moderation by admins.

Acceptance: sandbox payment end-to-end from portal to ledger to receipt; SMS adapter tested with a fake driver; message threads respect policies.

PHASE 7 — Hybrid desktop application (installable, fully offline, syncs with the cloud)

Goal: a Windows (and macOS/Linux) installer that runs on a machine with no internet and synchronises with the hosted server when online. Build two delivery modes and let me choose per school:

Mode 1 — Desktop client (lightweight). Package the PWA from Phases 4–5 in a native shell (Tauri preferred for size; Electron acceptable) with: `.msi` / `.exe` installer, auto-update, tray icon, native notifications, printer access for receipts/report cards, and the

server URL set at first launch. Data lives in the encrypted IndexedDB store. Best for individual teachers and bursars.

Mode 2 — Full local "campus node" (heavy). Package the whole Laravel app with **NativePHP for Desktop** (v2.x; confirm Laravel 13 support for the installed version first) running a local SQLite database, so *every* module works offline on that machine or on the school LAN:

1. Fix every item in `docs/upgrade/SQLITE_GAPS.md` so all migrations and queries run on SQLite.
2. First-launch wizard (bilingual): sign in to the cloud server → choose school → initial full download via `/api/v1/sync/pull` → done.
3. A background sync service (NativePHP scheduled task) pushes the local outbox and pulls changes using the same Phase 3 protocol and Phase 5 conflict rules; the local node is treated as one device.
4. Optional LAN mode: other computers on the school network open `http://<campus-node>:8000` in a browser when the internet is down.
5. Code-signed installers for Windows (Azure Trusted Signing or a certificate), auto-update channel, local encrypted backups.

Acceptance: fresh Windows machine → install → sign in once online → disconnect network → run a full school day (attendance, marks, payments, notices, printing) → reconnect → all changes appear on the cloud and other devices with no data loss; uninstall leaves no orphan data unless the user keeps a backup.

PHASE 8 — New features that make the product stand out

Deliver in this order, each fully bilingual, tested and available offline where marked (O):

1. **Cameroon academic templates:** francophone (séquences 1–6, trimestres, coefficients, moyennes /20, mentions, conseil de classe, bulletins officiels) and anglophone (Form 1–5, Lower/Upper Sixth, GCE O/A-Level grading) templates for grading scales, report cards and transcripts.
2. **Homework and assignments (O):** teachers post, students submit (files/photos), teachers grade into the gradebook.
3. **Online quizzes / CBT:** question bank, timed tests, auto-marking, results to the gradebook.
4. **Student ID cards and QR codes:** printable cards; QR scan for attendance at the gate (O) and library loans (O).

5. **Public admissions portal:** online application, document upload, entrance-exam scheduling, admission letters — bilingual.
6. **HR and payroll lite:** staff contracts, salary scales, monthly payslips (CNPS fields as configurable deductions).
7. **Transport:** routes, stops, vehicles, pupils per route, fees.
8. **Analytics:** early-warning list (attendance + grades + balance), class performance comparisons, exportable charts.
9. **AI assistant (optional, behind a feature flag):** draft report-card comments in EN/FR from marks, summarise a student's term, answer "how do I…" questions. Ask me which AI provider to use.
10. **Audit & compliance:** activity log viewer, data export per student, configurable retention.

PHASE 9 — Hardening, performance and release

1. Security review: OWASP top 10, rate limits, CSP headers, token scopes, encrypted backups, 2FA enforced for admins and accountants.
2. Performance: eliminate N+1 queries (the repo already has a query detector), add indexes for sync cursors, cache heavy dashboard queries, target < 300 ms server time on main pages with 2,000 students.
3. Load test with seeded data: 5 campuses, 10,000 students, 3 years of history.
4. Bilingual user documentation: admin, teacher, accountant and parent guides (PDF + in-app help), and a VPS deployment guide.
5. Release checklist: backup/restore rehearsal passes, `release-readiness` command green, version tagged, changelog in EN and FR.

Acceptance: all CI checks green, documented manual test scripts pass in both languages, installers published.

How to start (first message to your agent)

> "Read the CONTEXT and GLOBAL RULES above, then read `AGENTS.md`, `CLAUDE.md`, `.ai/rules`, `routes/web.php`, `app/Http/Middleware/SetApplicationLocale.php`, `resources/views/layouts/app.blade.php` and `app/Enums/Feature.php`. Summarise your understanding in 15 lines, then execute PHASE 0 step by step. Stop at the end of Phase 0 and report."

Sources

- NativePHP for Desktop on Packagist (v2.4.0, 22 Oct 2026, declares Laravel 10–13)
- NativePHP support policy (officially lists Laravel 11–12, hence the “confirm first” step)
- MTN MoMo Collection API in Cameroon (Kolonell, Aug 2026)
- CamPay payment API overview
- Skuul source: your uploaded `skuul-main.zip` (V3 `main` branch)